# Group 10 Final Report: Idiom Classification

**Rebecca Di Filippo, Leo Vugert, Timothy Pokani, Claire Nielsen**
{difilir,vugertl,pokanait,nielsc2}@mcmaster.ca

## 1 Introduction

For this project, the team has created software that identifies and classifies idiomatic expressions and literary devices in English text. Specifically, this task focuses on detecting and labelling the type of figurative language present in short text segments, including: metaphors, similes, personification, as well as a few others.

Understanding idiomatic and figurative language is a fundamental challenge in natural language processing because the intended meaning often cannot be derived directly from the literal meanings of the individual words. While humans can usually recognize these expressions using context and world knowledge, computational models struggle due to ambiguity, cultural conventions, and the subtle differences between similar devices (for example, discerning a metaphor vs. an idiom).

This task is significant because idiomatic language appears frequently in real world applications of natural language processing such as social media analysis, educational tools, and conversational agents. Misinterpreting these expressions can lead to incorrect or misleading system responses. Unlike standard sentiment analysis, which focuses primarily on emotional polarity, this task requires the software to perform semantic disambiguation and contextual reasoning. This goes beyond surface-level text classification, and requires the model to navigate linguistic ambiguity where the intended meaning cannot be derived directly from the literal meanings of individual words.

## 2 Related Work

The team found their dataset attached to a study done in 2022, which performed the same task. This study was incredibly helpful as a benchmark, as it tested for the same categories as this task. This was the first indication that BERT would perform quite well, as in this study it outperformed both Multinomial Naive-Bayes and Support Vector Machine models **?**. The BERT resultant F1-Score was 0.948, which this team would later surpass.

A study performed in 2009 examined the results of a combined classifier that uses both supervised and unsupervised models to examine literal vs. non-literal use of idiomatic expressions. This study found that the combined classifier was more accurate than the unsupervised classifier alone and the majority baseline; the combined classifier resulted in an error reduction of over 35% (**?**).

Research by **?** highlighted the success of the BERT model when applied to a text to analyse literary devices. Models such as GloVe and Word2Vec treat words as separate entities, while BERT takes in the surrounding context, leading to much better performance analysing literary devices. This paper reported an F1-Score of 0.85 resulting from applying BERT to detect metaphors on a corpus build from Project Gutenberg and other public literary anthologies.

A similar paper published in 2024 tested the BERT model on text classification, sentiment analysis, and and question answering. This paper found that BERT yielded an F1-Score on text classification of 0.915, a score on sentiment analysis of 0.885, and a score on question answering of 0.865. These results were all higher than their counterparts performed by recurrent neural networks (RNNs), supporting the conclusion that the BERT model would outperform RNN methods, at the cost of much longer training time and computational load (**?**).

A study was done by **?** that compared BERT and DistilBERT while performing sentiment analysis. It explains that by updating parameter weights used on the BERT model, DistilBERT can perform better on sentence classification when using a full set of parameters.

## 3 Dataset

### 3.1 Source and Collection

The dataset utilized for this project is the Potential Idiomatic Expression (PIE)-English Corpus, which is a labeled dataset designed for the classification of idiomatic expressions and literary devices. The data is publicly available via a GitHub repository and is derived primarily from the British National Corpus (BNC), which accounts for 96.9% of the samples, and UK Web Pages (UKWaC), which accounts for the remaining 3.1%. The original corpus was extracted by near-native English speakers and labeled by independent annotators to ensure accuracy.

### 3.2 Dataset Properties and Labels

The task is defined as a single-label classification problem where each text segment is mapped to one of ten primary literary device or idiomatic categories. The ten classes include: Euphemism, Hyperbole, Irony, Literal, Metaphor, Oxymoron, Paradox, Parallelism, Personification, and Simile.

A critical characteristic of this dataset is the significant class imbalance. As shown in Table 1, the distribution is heavily skewed toward Metaphors, while some categories, such as Irony and Oxymoron, represent less than 1% of the total data points. This distribution presents a notable challenge for model training, as the minority labels have very low frequency.

Table 1: Label Distribution in the PIE-English Corpus

| Label | Frequency | Label | Frequency |
|---|---|---|---|
| Metaphor | 72.7% | Oxymoron | 0.24% |
| Euphemism | 11.82% | Paradox | 0.56% |
| Simile | 6.11% | Hyperbole | 0.24% |
| Literal | 5.65% | Parallelism | 0.32% |
| Personification | 2.22% | Irony | 0.16% |

### 3.3 Preprocessing

The original source data was provided in a token-per-row CSV format. To make the data suitable for sentence-level classification and manual annotation, we implemented a Python script to perform the following preprocessing steps:

- **Sentence Reconstruction:** Tokens were joined to reconstruct full sentences from the tokenized rows.
- **Normalization:** All reconstructed sentences were converted to lowercase.
- **Random Sampling:** From the total 20,174 samples, we randomly sampled 1,200 data points to create a balanced mixture for our team's manual annotation phase.

### 3.4 Annotation Procedure and Reliability

Our team followed a structured annotation protocol using a shared spreadsheet interface. Each of the four team members independently annotated 300 unique data points, resulting in 1,200 total primary annotations. To measure Inter-Annotator Agreement (IAA), each member also re-annotated a subset of 45 data points originally labeled by a teammate.

The consistency of our annotations was evaluated using Cohen's Kappa for pairwise comparisons and Krippendorff's Alpha for overall group reliability. Our pairwise Cohen's Kappa scores ranged from 0.6293 to 0.8990, representing substantial to great agreement. The final Krippendorff's Alpha was 0.8790, which actually exceeded some individual pairwise scores, suggesting our group was highly aligned with the original corpus's "Gold Standard" labels

### 3.5 Data Observations

During the annotation process, we identified several interesting linguistic nuances. Some sentences presented "edge cases" where literal and metaphorical meanings overlapped, such as *"the worry at the back of her mind,"* which is metaphorical, appearing alongside the literal phrase *"she sat down beside her patient"*. Furthermore, because the corpus is British-centric, we encountered regional idioms such as *"not by a long chalk,"* which is the UK equivalent of the North American *"not by a long shot"*. These cultural variations and internal sentence ambiguities highlight the difficulty of the classification task for automated models.

## 4 Features

### 4.1 Model Inputs and Representation

The primary input for our models consists of reconstructed English sentences derived from the token-per-row CSV format of the PIE-English Corpus. During the preprocessing phase, these tokens are joined to form complete grammatical units, ensuring the models have access to the full sentential context necessary for disambiguating figurative language. To reduce the complexity of the input space and handle vocabulary sparsity, all text is normalized to lowercase before being fed into the embedding layers.

### 4.2 Representation Learning and Embeddings

Our team utilizes a combination of learned and pretrained embeddings. For our transformer-based models, we perform representation learning where the model learns task-specific embeddings through fine-tuning.This approach is vital because the meaning of an idiom often cannot be derived from individual word embeddings alone. By fine-tuning, the model develops a representation that prioritizes the semantic relationship between words in a figurative context.

### 4.3 Feature Engineering

For our final implementation, we shifted away from manual feature engineering to focus on automated representation learning. Rather than defining specific linguistic markers, we utilize contextualized word embeddings from a fine-tuned transformer model (DistilBERT) to capture the complex semantic relationships inherent in idiomatic language. This approach allows the neural network to internally learn the necessary features to distinguish between literal and figurative meanings without the need for manually defined rules. This is particularly critical for our task because the intended meaning of an idiom often cannot be derived from the literal definitions of individual tokens.

### 4.4 Feature Selection and Data Augmentation

The PIE-English Corpus presents a significant challenge due to its extreme class imbalance, with metaphors representing approximately 72.7% of the total samples. After evaluating our approach, we decided against performing specific feature selection or data augmentation. Instead, we utilized the entire dataset of 20,174 samples for training and evaluation to ensure the models were exposed to the maximum possible number of instances for minority labels, such as paradoxes and ironies. This decision ensures that our models learn from the natural distribution of the data while maintaining exposure to the rarest but arguably more relevant categories in the corpus.

## 5 Implementation

The simple baseline for our model was a majority class classifier. In the implementation, a class was initialized with the `most_frequent` strategy on sklearn's `DummyClassifier`. The data is loaded into training dataframes and testing dataframes. The training data is split into the sentences and labels, which is then loaded into the classifier. The test dataframe is then loaded into the predict function and then exported to a CSV. The most frequent label in our dataset is Metaphor.

Our simple trained model baseline used TF-IDF to vectorize the tokens, then trained a logistic regression model. The data was built very similarly to the simple baseline in terms of importing data, fitting the model, predicting, and exporting. However, to simplify the code, a pipeline was built that took in the data, vectorized it with TF-IDF, and then trained the logistic regression model as well at once.

The next model that was fine-tuned was the ModernBert Base model. Training, validation, and test CSVs were loaded into Pandas data frames, the labels were then encoded, and the data frames were passed into HuggingFace datasets. The model, tokenizer, and compute metrics were then initialized, parameters were passed through the training arguments, the trainer is initialized, and once the training was done, the model predicts and exports a CSV. Training was done on Google Colab using Nvidia T4 GPUs, after many hyperparameter tunings the learning rate was set to 5e-5, train batch to 16, evaluation batch to 32, 6 epochs, warm up ration of 01, weight decay to 0.01, and the best model was set to be loaded at the end. The model was then retrained with 8 epochs and included the validation data as training data and this produced our best results.

The following models were built similarly, just without the retraining with the validation data at the end. All the models we used were BERT models because they get a better understanding of the context of sentences.

The second model we trained was a Bert Base

Uncased model. The training arguments were learning rate as 2e-5, train batch size is 16, evaluation batch size is 16, 3 epochs, 0.01 weight decay, and the best model was loaded at the end.

The third model was a RoBERTa model with a learning rate of 1.5e-5, train batch size is 16, evaluation batch size is 32, 6 epochs, 0.01 weight decay, 0.1 warmup ratio, and the best model was loaded at the end. Fp16 was enabled to speed up training time and reduce memory usage.

The fourth model was DeBERTa, though it produced poor results at the beginning, after finetuning the parameters, the results improved exponentially. The learning rate was set to 1.5e-5, train batch was 16, evaluation batch was 32, 6 epochs, warm up ratio set to 0.1, weight decay was 0.01, the best model was loaded at the end, and bf16 and fp16 were enabled for faster and more efficient training.

Table 2: Hyperparameter Configurations for Transformer Models

| Model | LR | Train Batch | Eval Batch | Epochs | Weight Decay | Warmup |
|---|---|---|---|---|---|---|
| ModernBERT Base | 5e-5 | 16 | 32 | 6 | 0.01 | 0.1 |
| ModernBERT (Retrained) | 5e-5 | 16 | 32 | 8 | 0.01 | 0.1 |
| BERT Base Uncased | 2e-5 | 16 | 16 | 3 | 0.01 | - |
| RoBERTa | 1.5e-5 | 16 | 32 | 6 | 0.01 | 0.1 |
| DeBERTa | 1.5e-5 | 16 | 32 | 6 | 0.01 | 0.1 |

## 6 Results

The simple baseline model received an accuracy score of 0.73, but a Macro F1 score of 0.08. Since the simple baseline model is majority class classifier, it accurately predicts 73

The logistic regression model had an accuracy of 0.87 and a Macro F1 score of 0.9. This one was much higher because the F1 score of classes other than metaphors were way higher than in the simple baseline model (where they were 0). By treating the text as a sparse "bag of words" using TF-IDF, the model successfully exploited specific vocabulary artifacts and trigger words unique to the minority classes. This simpler linear approach effectively isolated lexical cues without being overwhelmed by broader sentence context.

The BERT Base Uncased model achieved an accuracy of 0.95 and a Macro F1 score of 0.63. As a 110-million parameter model originally pre-trained over four days on 16 TPUs using the BooksCorpus and English Wikipedia (roughly 16GB of text), it is the oldest architecture in our deep learning lineup. While the 0.95 accuracy appears strong on the surface, the 0.63 Macro F1 reveals that it still partially fell victim to majority class bias. Its standard self-attention mechanism lacked the semantic depth required to confidently differentiate the highly ambiguous minority classes, frequently defaulting to the dominant "Metaphor" label when it encountered complex or unfamiliar sentence structures.

The RoBERTa model showed significant improvement, achieving an accuracy of 0.97 and a Macro F1 score of 0.88. RoBERTa is a 125-million parameter architecture that was pre-trained on a much larger 160GB corpus (including CC-News and OpenWebText) using 1024 V100 GPUs. By removing BERT's next-sentence prediction objective and utilizing dynamic masking during its massive pre-training phase, RoBERTa captured much richer and more robust semantic representations. This broader world knowledge allowed it to understand the underlying context of minority classes like Irony and Paradox much better than base BERT, raising its Macro F1 substantially.

The DeBERTa model achieved an accuracy of 0.96 and a highly impressive Macro F1 score of 0.94. This roughly 140-million parameter model was pre-trained on 78GB of data using 64 V100 GPUs, but its true advantage lies in its architecture. DeBERTa utilizes a unique "disentangled attention" mechanism that computes a word's content and its relative position in a sentence separately. This gave the model a massive structural advantage in parsing complex figurative language. By explicitly understanding how words relate to each other spatially, DeBERTa excelled at identifying highly structural literary devices such as Similes and Parallelism which allows it to easily outperform the logistic regression baseline.

Finally, ModernBERT was our best-performing model, achieving a near-perfect accuracy of 0.98 and a Macro F1 of 0.97. ModernBERT is a 149-million parameter model pre-trained on an unprecedented 2 trillion tokens of web data, code, and scientific literature. Benefiting from state-of-the-art architectural upgrades like Rotary Position Embeddings (RoPE) and unpadded Flash Attention, it possesses an incredibly deep semantic understanding. This immense pre-training knowledge, combined with our strategy of retraining the model on the full, combined dataset, allowed ModernBERT to completely overcome the severe class imbalance. It proved that with a modern enough architecture, deep contextual models can extract the subtle, underlying meanings of the rarest figurative language

classes without needing to rely on surface-level trigger words.

Table 3: Model Performance Evaluation on Figurative Language Classification

| Model | Accuracy | Macro F1 |
|---|---|---|
| Majority Class Baseline | 0.73 | 0.08 |
| Logistic Regression (TF-IDF) | 0.87 | 0.90 |
| BERT Base Uncased | 0.95 | 0.63 |
| RoBERTa | 0.97 | 0.88 |
| DeBERTa | 0.96 | 0.94 |
| ModernBERT | **0.98** | **0.97** |

## 7 Evaluation

### 7.1 Evaluation Strategy

Our evaluation strategy is designed to assess the model's ability to generalize across a highly imbalanced multi-class dataset. We utilize a stratified sampling approach for our data splits to ensure that the minority classes, such as Irony and Oxymoron, are represented in the validation and test sets in proportions consistent with the overall corpus. The primary goal is to outperform a simple majority-vote baseline, which would achieve high accuracy by simply predicting "Metaphor" for every instance.

### 7.2 Train, Validation, and Test Splits

The dataset, consisting of 20,174 samples, was divided into training, validation, and test sets using an 80/10/10 split. The sizes of these splits are as follows:

- **Training Set:** 16,139 samples (80%) used for model parameter optimization and feature learning.
- **Validation Set:** 2,017 samples (10%) used for hyperparameter tuning and preventing overfitting.
- **Test Set:** 2,018 samples (10%) held out for the final evaluation of model performance.

The label distribution remains consistent across these splits, with Metaphors accounting for roughly 72.7

### 7.3 Evaluation Metrics

Due to the extreme class imbalance identified during our initial analysis, accuracy alone is an insufficient metric for this task. Instead, we prioritize the following metrics:

- **Macro-Averaged F1-Score:** This is our primary metric, as it treats all ten classes equally, regardless of their frequency. This ensures that the model is penalized heavily if it fails to correctly identify the minority literary devices like Paradox or Irony.
- **Weighted F1-Score:** We use this to understand the model's overall performance while accounting for the high prevalence of metaphors in real-world English text.
- **Precision and Recall per Class:** We generate a classification report to examine specific errors, such as the model's ability to distinguish between metaphors and similes or metaphors and literal statements.
- **Confusion Matrix:** This qualitative tool is used to visualize which classes are most frequently confused, providing insight into the semantic overlap between similar figurative devices.

## 8 Comparison to Others

| Model | Accuracy | F1 Score |
|---|---|---|
| roberta (team) | 0.97 | 0.88 |
| bert (team) | 0.95 | 0.63 |
| modern-bert (team) | 0.98 | 0.97 |
| deberta (team) | 0.96 | 0.94 |
| Majority Vote (team) | 0.73 | 0.08 |
| modern-bert (AK) | 0.98 | 0.97 |
| roberta-base (AK) | 0.97 | 0.94 |
| Majority Vote (AK) | 0.73 | 0.08 |
| roberta-base (KO) | 0.90 | 0.35 |
| bert (KO) | 0.86 | 0.33 |
| BoW Classifier (KO) | 0.85 | 0.43 |

Table 4: Comparison of Group 10's Results with Peers' Results.

The best-performing model produced by peers was a fine-tuned modernBERT-base model by Arjun Karthik. This model resulted in an accuracy of 0.98 and Macro F1 of 0.97. This is an incredible result, and is the best-performing result on the team's Codabench page. The modernBert-base model run by the team produced was close behind, resulting in an accuracy of 0.96 and F1 score of 0.94.

Within Arjun's results, we also saw a fine-tuned roberta-base model performed quite well, producing an accuracy of 0.97 and F1 of 0.94. This model once again outscored the team's roberta-base, which produced an accuracy of 0.97, but an F1 score of 0.88.

Arjun's Majority Vote Baseline model saw an accuracy of 0.73, and an F1 score of 0.08. This makes sense, since the majority class in the dataset is Metaphor, making up 72.7% of the total data points. Since the majority vote makes up so much of the class, it makes sense that accuracy would be so high, but precision and therefore F1 score would be so low.

Both of Arjun's zero-shot models performed in the same ballpark: the facebook/bart-large-mnli produced an accuracy of 0.41 and F1 of 0.09; the MoritzLaurer/DeBERTa-v3-base-mnli-fever-anli produced an accuracy of 0.42 and F1 score of 0.08. Since zero-shot models classify using no training data, it makes sense that these models perform far worse than trained models.

Another peer that worked with our classification task was Kenneth Ong. Kenneth's best-performing model produced an accuracy of 0.90, however this was accompanied with an F1 score of 0.35, markedly lower than other results. His Bag-of-Words classifier actually produced a higher F1 score of 0.43, along with a lower accuracy of 0.85. Overall, these results were outscored by our team.

Kenneth ran a bert-base-uncased model that produced an accuracy of 0.86 and F1 score of 0.33. This was outscored in both categories by our team's bert-base-uncased model, which produced an accuracy of 0.95 and F1 score of 0.63.

## 9 Error Analysis

To systematically examine the errors made by our models, we analyzed the confusion matrices generated for each transformer model (ModernBERT, DeBERTa, RoBERTa, and BERT Base Uncased). These matrices, detailed in the appendix, provide a quantitative view of misclassifications across the ten figurative language classes.

Common error patterns observed include frequent confusion between Metaphor and Literal expressions, as well as between Metaphor and Euphemism. For instance, in the ModernBERT model, out of 2901 correct Metaphor predictions, 19 were misclassified as Euphemism and 10 as Literal. This suggests semantic overlap where the boundary between figurative and literal meanings is ambiguous. Minority classes like Irony and Hyperbole showed higher error rates; for example, the BERT Base Uncased model correctly predicted 0 instances of Hyperbole, often defaulting to Metaphor.

Qualitatively, models struggled with sentences exhibiting subtle contextual cues. For example, the phrase "the worry at the back of her mind" was sometimes misclassified as Literal instead of Metaphor due to its metaphorical nature being context-dependent. Similarly, British idioms like "not by a long chalk" posed challenges for models not exposed to regional variations.

ModernBERT excelled across all classes, with high accuracy even on minority labels (e.g., 6 correct Irony predictions), demonstrating its robustness to class imbalance. In contrast, older models like BERT Base Uncased performed poorly on rare classes, highlighting the benefits of advanced architectures and larger pre-training corpora.

To address these issues, future work could incorporate data augmentation techniques, such as generating synthetic examples for minority classes, or loss functions like focal loss to emphasize hard-to-classify instances. We recommend prioritizing modern transformer models like ModernBERT for similar tasks, while ensuring diverse training data to capture cultural and linguistic variations.

## Conclusions

Summarize your work briefly. Reflect on your implementation and areas for improvement. What do you suggest people work on next (mentioned also in item 7)? This should likely be less than 0.25 pages. BECCA

## 10 Team Contributions

The members of Team 10 each took on specialized roles to ensure the successful execution of both the technical code and documentation aspects of the project.

- **Rebecca Di Filippo**: Created repository and main task was helping create the final report. Also assisted choosing coding methods for models.

- **Claire Nielsen**: Led the documentation efforts, including writing the final report and ensuring that all sections were cohesive and well-structured.
- **Tim Pokanai**: Implemented 2 of the 4 models, and helped right the implementation and error analysis section.
- **Leo Vugert**: Implemented 2 of the 4 models, and helped right the implementation and error analysis section.

# A Confusion Matrices

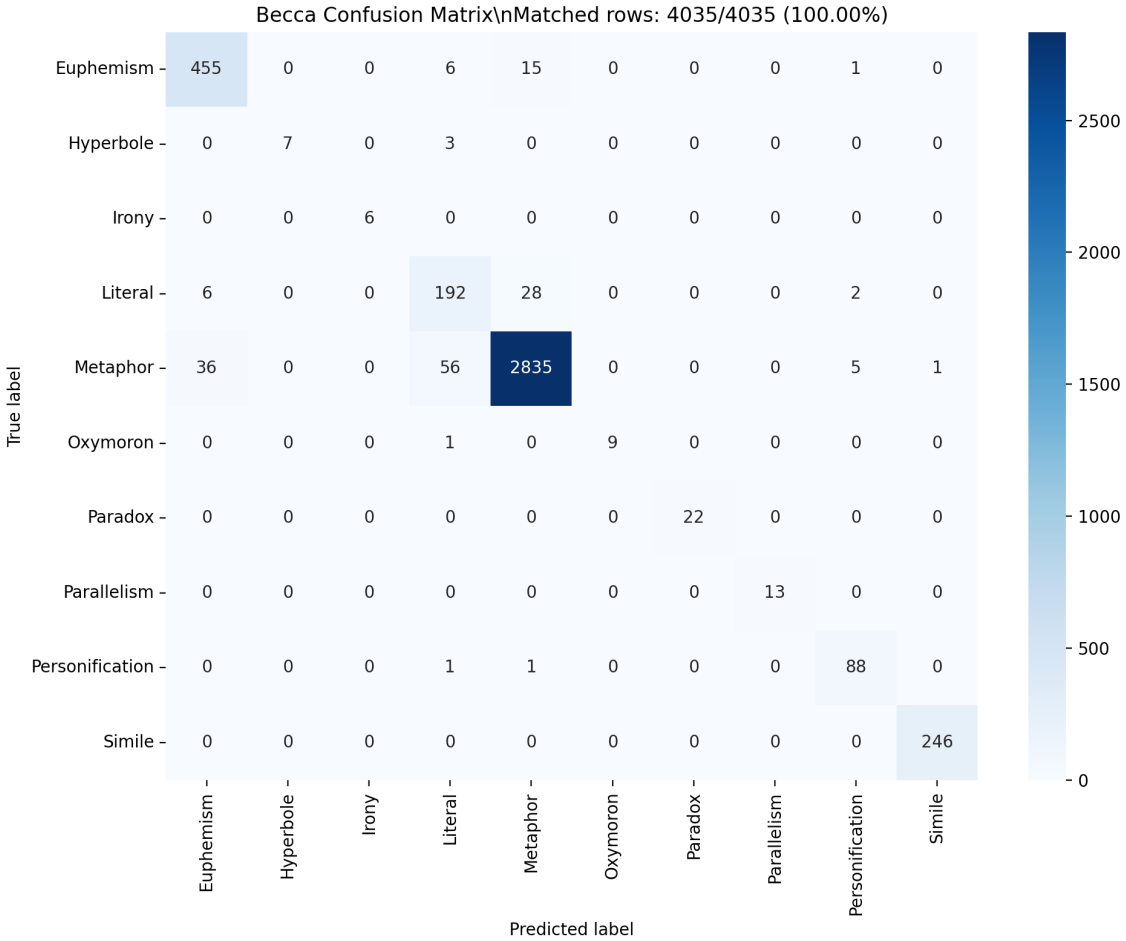


Figure 1: Confusion Matrix for Becca's Model

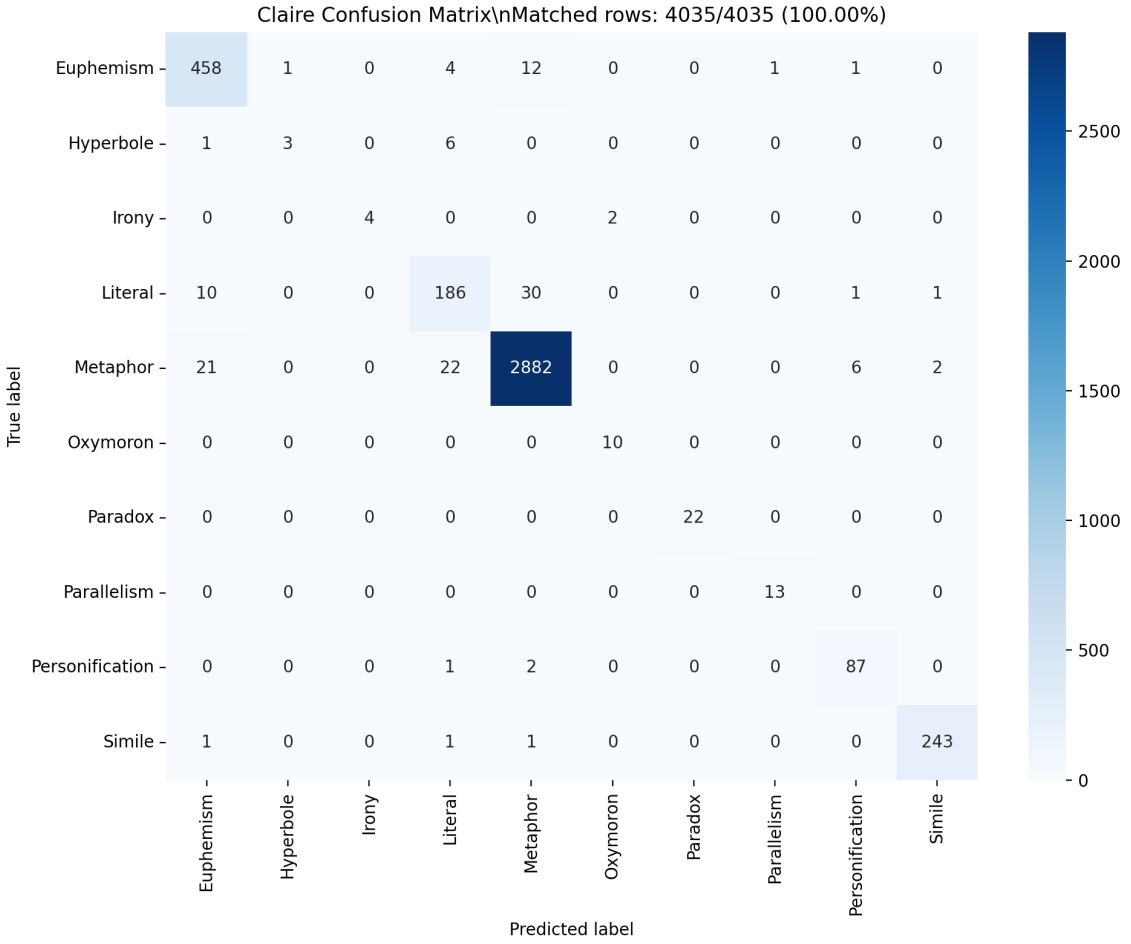


Figure 2: Confusion Matrix for Claire's Model

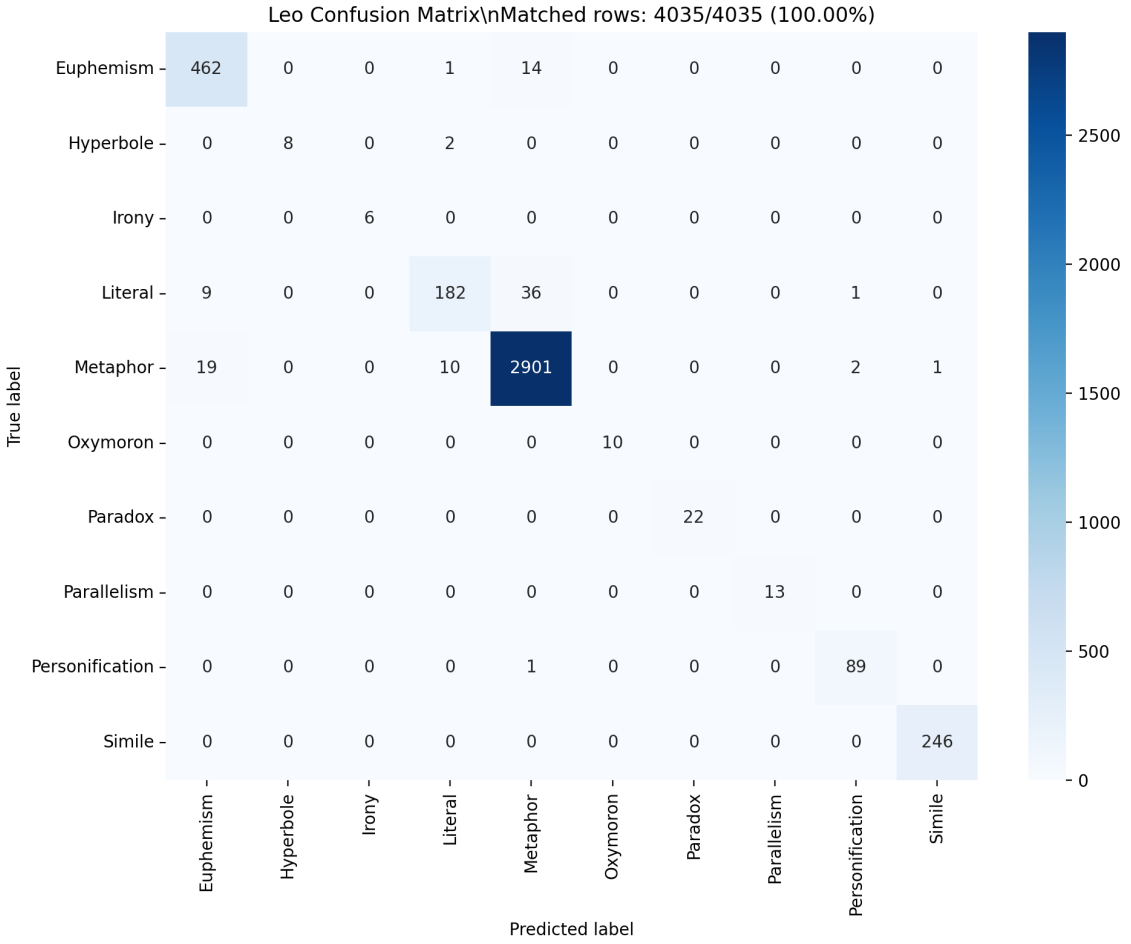


Figure 3: Confusion Matrix for Leo's Model

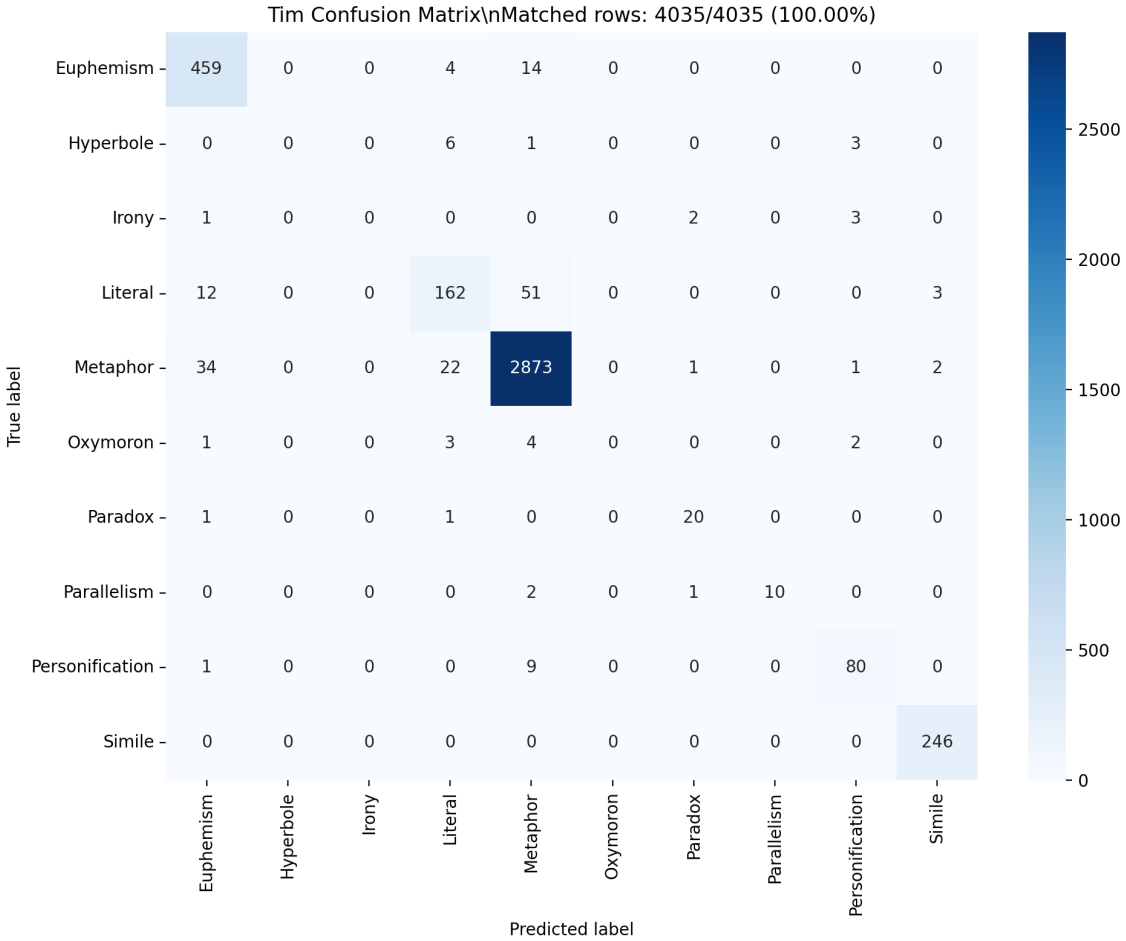


Figure 4: Confusion Matrix for Tim's Model